

Assignment: Temporary Tables & Views - Hands-on & Scenario-Based Practice

Note: Work inside `college_db.academics`. Write and test every statement in pgAdmin's Query Tool. Observe real behavior (especially session scope of TEMP tables) before answering.

Setup Reference

Use the same tables from previous days:

Table	Key Columns
academics.departments	dept_id, dept_name
academics.students	student_id, full_name, email, dept_id, cgpa, age
academics.courses	course_id, course_name, dept_id, credits
academics.enrollments	student_id, course_id, enroll_date, grade

Q1. Create a Temporary Table from Query

Create a TEMP table named `temp_top_students` that contains `student_id`, `full_name`, and `cgpa` for all students with `cgpa >= 7.5`. Then `SELECT` from it ordered by `cgpa DESC`.

-- Write your `CREATE TEMP TABLE` and `SELECT` here:

```
CREATE TEMP TABLE temp_top_students AS
```

```
SELECT
```

```
    student_id,
```

```
    full_name,
```

```
    cgpa
```

```
FROM academics.students
```

```
WHERE cgpa >= 7.5;
```

```
SELECT *
```

```
FROM temp_top_students
```

```
ORDER BY cgpa DESC;
```

Q2. TEMP Table + JOIN

Using the `temp_top_students` table from Q1 (recreate if needed), join it with `enrollments` and `courses` to list each top student's courses and grades.

-- Write your `SELECT` statement here:

```
SELECT
```

```
    t.student_id,
```

```
    t.full_name,
```

```
    c.course_name,
```

```
    e.grade
```

```
FROM temp_top_students t
```

```
JOIN academics.enrollments e
ON t.student_id = e.student_id
JOIN academics.courses c
ON e.course_id = c.course_id
ORDER BY t.full_name;
```

Q3. Session Scope of TEMP Tables

(a) Create a TEMP table temp_demo with a single column id and insert one row.

(b) In a comment, explain what happens if you open a brand-new Query Tool tab (new session) and try to SELECT from temp_demo.

(c) DROP the temp table in the original session.

-- (a) statements, (b) explanation, (c) DROP:

A)

```
CREATE TEMP TABLE temp_demo
(
    id INT
);
```

```
INSERT INTO temp_demo
VALUES (1);
```

B)

If I open a new Query Tool and run

```
SELECT * FROM temp_demo;
```

PostgreSQL will give an error saying the table does not exist.

This is because TEMP tables are available only in the session in which they were created.

C)

```
DROP TABLE temp_demo;
```

Q4. Create a Simple View

Create a view academics.v_students_with_dept that returns student_id, full_name, cgpa, and dept_name (LEFT JOIN so students without department still appear).

-- Write your CREATE VIEW statement here:

```
CREATE VIEW academics.v_students_with_dept AS
SELECT
    s.student_id,
    s.full_name,
    s.cgpa,
    d.dept_name
FROM academics.students s
LEFT JOIN academics.departments d
ON s.dept_id = d.dept_id;
```

Q5. Query and Filter a View

Using the view from Q4, list students whose cgpa is above 8.0, ordered by cgpa descending.

-- Write your *SELECT* statement here:

```
SELECT *  
FROM academics.v_students_with_dept  
WHERE cgpa > 8.0  
ORDER BY cgpa DESC;
```

Q6. Aggregated View

Create a view academics.v_dept_stats that shows for every department: dept_name, student_count, and avg_cgpa (rounded to 2 decimals). Include departments with zero students.

-- Write your *CREATE VIEW* statement here:

```
CREATE VIEW academics.v_dept_stats AS  
SELECT  
    d.dept_name,  
    COUNT(s.student_id) AS student_count,  
    ROUND(AVG(s.cgpa), 2) AS avg_cgpa  
FROM academics.departments d  
LEFT JOIN academics.students s  
ON d.dept_id = s.dept_id  
GROUP BY d.dept_name;
```

Q7. Complex Multi-join View

Create a view academics.v_enrollment_report that returns:

student_name, student_dept, course_name, course_dept, grade, enroll_date

Join students → enrollments → courses → departments (twice — once for student dept, once for course dept).

-- Write your *CREATE VIEW* statement here:

```
CREATE VIEW academics.v_enrollment_report AS  
SELECT  
    s.full_name AS student_name,  
    sd.dept_name AS student_dept,  
    c.course_name,  
    cd.dept_name AS course_dept,  
    e.grade,  
    e.enroll_date  
FROM academics.students s  
JOIN academics.enrollments e  
ON s.student_id = e.student_id  
JOIN academics.courses c  
ON e.course_id = c.course_id  
LEFT JOIN academics.departments sd  
ON s.dept_id = sd.dept_id
```

```
LEFT JOIN academics.departments cd
ON c.dept_id = cd.dept_id;
```

Q8. CREATE OR REPLACE VIEW

Modify the view academics.v_students_with_dept (from Q4) to also include the email column. Use CREATE OR REPLACE VIEW. Then query the view to confirm the new column appears.

-- Write your CREATE OR REPLACE and verification SELECT here:

```
CREATE OR REPLACE VIEW academics.v_students_with_dept AS
SELECT
```

```
    s.student_id,
    s.full_name,
    s.email,
    s.cgpa,
    d.dept_name
```

```
FROM academics.students s
```

```
LEFT JOIN academics.departments d
```

```
ON s.dept_id = d.dept_id;
```

Q9. Updatable View Test

- Create a simple view academics.v_student_cgpa that selects only student_id, full_name, cgpa from students (no joins, no aggregates).
- UPDATE the view to change the cgpa of one student.
- Verify the change appears in the base students table.
- In 1-2 sentences explain why this view is updatable while v_enrollment_report is not.

-- (a)–(d) statements and explanation:

Q10. TEMP Table for Multi-step Analysis

Using only TEMP tables (no CTEs), perform these steps:

- Create temp_enroll_count → student_id + number of courses enrolled.
- Create temp_high_grades → student_id + count of A/A+ grades.
- Join both temp tables with students to produce a final report: full_name, courses_taken, high_grade_count.

Include students with zero enrollments (show 0s).

-- Write all statements here:

Q11. View vs TEMP vs CTE Decision

For each scenario below, write which tool you would choose (TEMP table / View / CTE) and one short reason:

- A daily dashboard query that many users will run — needs consistent logic.
- A one-time complex report with 4 intermediate steps inside a single query.
- An ETL-style process that builds intermediate datasets, indexes them, and runs multiple follow-up queries in the same session.

-- (a), (b), (c) answers:

Q12. Cleanup and Best Practice

- (a) DROP the temporary tables you created in this assignment.
- (b) DROP the views you no longer need (or keep the useful ones — your choice).
- (c) Write a short note (2-3 sentences) on when you would prefer a permanent table + scheduled job over a TEMP table for intermediate results.

-- (a), (b) *DROP statements* and (c) *note*:

Note: Useful views can stay in the schema — Next (Stored Procedures & UDFs) will often *SELECT* from views. *TEMP* tables will be gone when your session ends.